

零基础 Java CRUD 30 天落地小说 RAG 求职项目（纯云端 API、无本地大模型、可跳槽）

前置全局说明（必读，打消所有顾虑）

1. 你的专属适配条件

- 自身基础：2 年半 Java，只会业务 CRUD，Redis、多线程、AI 零基础，不懂任何算法
- 项目架构：全程只用云端通义千问 API，不装本地大模型、不用显卡、不用折腾 Ollama
- 技术极简阉割：只学最低够用版，不学分布式、高并发锁、复杂调优
- 每日时长：固定 1.5~2 小时，下班后即可完成，不熬夜
- 最终产出：可 Postman 演示、可写简历、可面试口述完整项目，差异化网文 RAG（碾压普通客服 RAG）

2. 全局固定技术栈（全部 CRUD 级别）

- 核心框架：SpringBoot3.3 + JDK17（必须 17，一键创建项目）
- AI 调用：Spring AI Alibaba（阿里云通义千问云端 API，仅需 1 个密钥）
- 文件解析：Apache Tika（一键解析 txt 小说，不用自己读写复杂文件）
- 向量数据库：Milvus(Docker 一键启动，当成特殊 MySQL，只学新增+查询)
- 中间件：Redis(Docker 一键启动，只学 String 缓存存取值，零基础够用)
- 工具：Docker Desktop、IDEA2024、Postman、阿里云 DashScope 密钥

3. 项目通俗定义（面试背诵版）

这是一个私有小说文风仿写系统：上传网文 txt→自动拆分段落→调用千问 API 打标签（爽点/文风/人设）+转数字向量→存入向量库；用户输入写作需求，自动匹配同款小说片段，带入参考文风，调用千问生成统一风格小说，解决 AI 写文人设崩塌、文风跑

偏问题。

4. 全局规则

每天分：今日任务 + 分步操作 + 复制代码/命令 + 验收标准 + 当日避坑，照着复制即可，不用自己动脑写逻辑

第一周：环境零基础搭建+AI 最简单调用（Day1-Day7） 【零基础打底，不学复杂知识】

Day1：电脑环境统一配置（2h）

目标：装好所有软件，统一版本，杜绝后续报错

1. 卸载电脑旧 JDK，安装 JDK17（阿里云免费一键安装包）
2. 安装 Docker Desktop（windows/mac 一键下一步，默认配置，不用改参数）
3. 安装 IDEA2024 社区版、Postman
4. 注册阿里云账号，开通【DashScope 通义千问】，复制 API-key（保存记事本，全局只用这一个密钥）

验收：拿到千问 API 密钥，Docker 能正常打开运行

避坑：不要升级 Docker 高级功能，默认基础版够用

Day2：创建纯净 SpringBoot3 项目+依赖导入（1.5h）

1. spring 官网创建项目：SpringBoot3.3.2、JDK17、Maven
2. pom 复制导入 4 个核心依赖（复制粘贴即可）：SpringAlibaba、tika 文件解析、Redis、Milvus JavaSDK

验收：项目启动无报错，依赖全部下载完成

当日知识点：只懂 yml 配置文件修改，和以往 springboot 项目无区别

Day3：yaml 配置千问密钥，写第一个 AI 对话接口（1.5h）

1. application.yml 粘贴千问密钥配置，复制即可
2. 编写 Controller 接口：传入文字，调用云端千问，返回 AI 回答
3. Postman 调用接口测试：输入“写一段都市打脸短句”，成功返回内容

验收：接口通，能调用云端 AI，本机无任何模型

通俗理解：你只是用 Java 发 http 请求，调用阿里云现成 AI，和你调第三方支付接口一模一样

Day4: Docker 一键启动 Redis，学习 Redis 极简 CRUD (1.5h)

1. 复制 docker 命令，一键拉起 Redis 容器（无需改配置）
2. yml 配置 Redis 连接
3. 写测试接口：Redis 存字符串、取字符串（只学这两个用法，足够项目使用）

验收：能存 key-value、能取值，不用学分布式锁、过期策略

面试话术储备：项目使用 Redis 缓存高频写作提问结果，减少 AI 接口重复调用，降低费用

Day5: Docker 一键启动 Milvus 向量库 (1.5h)

1. 复制固定 docker 命令，一键拉起 Milvus 向量库（专门存小说向量的数据库）
2. yml 配置 Milvus 地址端口
3. 理解通俗概念：Milvus=特殊 MySQL，可以根据文字相似度查询数据

验收：Milvus 容器正常运行，连接无报错

避坑：不用懂向量索引、算法，全部用默认参数

Day6: 极简线程池入门（项目专用，不学并发原理） (1.5h)

1. 复制固定线程池配置类，直接放入项目
2. 使用场景：批量导入小说不卡死接口，异步处理文件
3. 实操：写一个异步打印测试，看懂@Async 注解用法即可

验收：能异步执行任务，接口不会阻塞卡顿

说明：不学 volatile、锁、CAS，只用项目现成异步线程池，面试够用

Day7: 第一周复盘+概念背诵 (1.5h)

背诵 4 个面试极简名词（必考，背会即可）

1. API 调用：Java 发 http 请求，调用阿里云现成大模型
2. Embedding：文字转一串数字，方便机器判断两段文字像不像
3. RAG：先找参考小说，再给 AI 仿写，防止 AI 乱写

4. 向量库：按文字相似度查询的数据库，替代 MySQL 模糊查询

第一周整体成果：环境全好、AI 可调用、Redis/向量库可用、异步功能可用

第二周：小说文件处理+切片+打风格标签（Day8-Day14）

【纯 CRUD+字符串操作】

本周核心：全部是文件读取、字符串切割、数据库入库，和业务 CRUD 完全一致

Day8：整合 Tika 工具，实现上传 txt 小说接口（1.5h）

1. 编写文件上传 Controller，接收前端 txt 文件
2. Tika 工具一键读取小说全部文字，自动过滤空格、换行、广告水印

验收：上传一本网文 txt，后台打印清洗后的纯正文

Day9：自研网文专属切片工具类（核心项目亮点）（1.5h）

1. 复制现成切片代码：自动按照 350 字切割，优先不切断人物对话、剧情段落
2. 区别普通项目：普通 RAG 均等切割，本项目适配网文，面试加分

验收：整本小说切成几百条 350 字左右小段，不割裂对话

Day10：调用千问 API，给小说片段打标签（1.5h）

标签固定 5 类（存入数据库字段）：文风类型、爽点类型、剧情节奏、人物语气、场景

1. 固定 Prompt 模板复制到代码
2. 每一段小说，调用千问返回 json 标签

验收：一段都市打脸文，自动打上【都市战神、当众打脸、快节奏、男主高冷、酒店宴会】标签

Day11：调用千问 Embedding 接口，文字转向量（1.5h）

1. 封装工具类：传入文本，云端返回浮点数组（向量数字）
2. 无计算、无算法，纯调用接口接收数据

验收：文本成功转为 1024 位数字向量

Day12：Milvus 创建小说专属表结构（1.5h）

1. 代码创建 Milvus 表：id、原文片段、标签 json、向量值、小说来源
2. 等同于 MySQL 建表，字段 CRUD

验收：向量库生成 novel_rag 数据表

Day13：完整链路：切片+打标签+转向量+批量入库（1.5h）

1. 整合 service 层代码，异步批量入库
2. 防止重复入库：新增文本 md5 去重

验收：上传一本 10 万字网文，自动全部入库向量库

Day14：第二周调试+bug 修复（1.5h）

优化：过长文本过滤、无效水文过滤、入库异常捕获

第二周成果：小说可以一键上传，自动处理完毕存入向量库

第三周：RAG 检索组装+小说生成接口开发（Day15-Day21）【项目核心业务】

Day15：实现写作需求向量检索接口（1.5h）

1. 用户输入写作需求：比如“写商场反派嘲讽男主打脸剧情”
2. 需求转向量 + 标签筛选，从库中召回 Top5 相似小说片段
3. 实现：向量相似度+标签混合检索（项目面试核心亮点）

验收：检索出来全部是同款打脸文风片段

Day16：组装 RAG 专属 Prompt 模板（固定仿写模板，复制即用）（1.5h）

固定指令：严格模仿参考片段用词、短句节奏、对话语气、爽点逻辑，禁止 AI 通用文笔，保留原书人设

Day17：开发章节生成主接口（1.5h）

完整链路代码整合：用户写作需求→检索参考片段→拼接 prompt→调用千问 max 生成章节

验收：调用接口，产出和入库小说一模一样文风章节

Day18: 新增 Redis 缓存优化 (1.5h)

1. 高频写作需求, 缓存检索结果+AI 返回内容
2. 减少云端 API 调用次数, 节省 token 费用

Day19: 新增全局异常+AI 接口熔断降级 (1.5h)

引入 Resilience4j 极简配置: 千问接口超时、限流后, 返回友好提示, 项目工程化加分

Day20: 实现 SSE 流式输出 (打字机效果) (1.5h)

后端流式返回文字, 前端实现打字机写小说, 项目演示颜值拉满, 面试加分

Day21: 第三周全链路联调 (1.5h)

完整流程测试: 上传小说→入库→提问写作→仿写成功, 全程无报错

第四周: 项目优化+简历打磨+面试话术+查漏补缺 (Day22-Day30) 【直接跳槽使用】

Day22: 新增小说全局人设库 (解决长篇人设崩坏) (1.5h)

新增 MySQL 人设表: 存储主角性格、口头禅、世界观、修炼等级, 生成章节强制读取人设, 项目拔高亮点

Day23: 多模型切换配置编写 (1.5h)

yml 编写两套配置: 云端千问 / 预留 Ollama 本地模型配置 (仅配置, 不用实操搭建), 面试回答: 项目支持一键切换本地私有化部署

Day24: 项目参数调优 (1.5h)

调整切片大小、检索相似度阈值、AI 温度参数, 文风更稳定

Day25: 整理项目接口文档+Postman 测试用例 (1.5h)

整理 5 个核心接口: 文件上传、小说入库、章节生成、人设新增、缓存清空

Day26: 打磨简历项目描述 (直接复制使用) (1.5h)

【简历成品文案】基于 SpringBoot3+SpringAlibaba+Milvus+Redis 搭建云端 RAG 网文仿写系统，适配企业云端 AI 业务架构；实现小说 txt 自动清洗、网文友好切片、爽点文风 AI 标签化、向量混合检索能力；依托通义千问云端 API 完成章节仿写，通过 Redis 缓存降低接口调用成本，线程池异步处理文件入库，解决通用 AI 文风跑偏、长篇人设崩塌问题，适配网文连载业务。

Day27：高频面试题背诵（专属零基础版）（2h）

整理 15 道必背面试题+标准答案，全部口语化，不用原理

Day28：补齐短板话术（掩盖你不懂并发 Redis）（1.5h）

固定面试回答：日常业务以 CRUD 开发为主，本 AI 项目针对性落地轻量化技术：使用线程池异步批量处理小说文件避免接口阻塞，使用 Redis 做 AI 结果缓存降本，仅掌握业务够用级中间件用法，复杂高并发分布式正在持续学习。

Day29：项目完整演示演练（1.5h）

从头到尾演示一遍：上传小说→提问仿写→产出同文风小说，熟练流畅

Day30：整体复盘+代码归档（1.5h）

代码注释完善、删除无用代码、归档 Docker 命令、密钥配置文档，项目完工

全局避坑终极清单（专门给零基础 CRUD 开发者）

1. 绝对不要本地装 Ollama、本地大模型，纯云端 API 最简单
2. Redis 只学 String 存取值，不学任何高级功能
3. Milvus 只学新增、相似度查询，不学索引调优、分片集群
4. 线程池只用配置好的模板，不学并发底层原理
5. 所有 AI 能力，全部调用云端接口，自己不写任何算法、矩阵、神经网络代码

最终项目竞争力总结

1. 适配你的底子：全程 CRUD、文件操作、接口调用，无高深代码
2. 区别烂大街项目：不是客服问答，是垂直网文仿写 RAG，面试官耳目一新
3. 贴合企业现状：企业线上全部云端 API 调用，本地模型仅内网涉密使用，完全贴合生产业务

4. 补齐短板：项目自带 **Redis**、异步线程、AI 业务能力，完美弥补你只会纯 **CRUD** 的短板

|（注：部分内容可能由 AI 生成）